- You can customize the OpenAPI description of your endpoints by adding metadata. You can provide tags, for example, by calling `WithTags()` on an endpoint and specify that an endpoint returns a type `T` with a `201` status code using `Produces<T>(201)`. Adding metadata improves your API's OpenAPI description, which in turn improves tooling such as Swagger UI.
- You can use NSwag to generate a C# client from an OpenAPI description. This approach takes care of using the correct paths to call the API, substituting parameters in the path, and serializing and deserializing requests to the API, removing much of the boilerplate associated with interacting with an API.
- You can add code generation to your project by using Visual Studio or the .NET API tool or by making manual changes to your project. Visual Studio and the .NET tool automate downloading the OpenAPI description to your local project and adding the necessary NuGet packages. You should update the NuGet packages to the latest versions to ensure that you have the latest bug or security fixes.
- NSwag automatically generates a C# method name on the main client class for each endpoint in the OpenAPI description. If the endpoint's OperationID is missing, NSwag generates a name, which may not be optimal. You can specify the OperationID to use for an endpoint in your OpenAPI description by calling `WithName()` on the endpoint.
- You can customize the client NSwag generates by adding an `<Options>` element inside the `<OpenApiReference>` in your .csproj file. These options are specified as command-line switches such as `/JsonLibrary:SystemTextJson`. You can change many things about the generated code with these switches, such as the serialization library to use and whether to generate an interface for the client.